

測試與驗證指引

本指引介紹如何進行端到端整合測試、驗證實體路徑 MTU 限制、使用 `tshark` /Wireshark 擷取並解密加密的 QUIC 流量，以及核對傳輸的統計數據。

1. 本機測試

在同一台機器上同時運行伺服器與用戶端，以驗證編譯狀態、NALU 解析器與分片重組器的完整運作流程。

步驟 1：生成 TLS 安全憑證

確保您已生成連線所需的自簽署憑證與金鑰檔案：

```
mkdir -p certs
openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 \
  -keyout certs/server.key -out certs/server.crt \
  -subj "/CN=localhost"
```

步驟 2：啟動伺服器端

首先啟動接收端伺服器監聽 `4433` 埠號，並指定重組後的視訊輸出檔名：

```
cd build
./moq_server 0.0.0.0 4433 output_test.h265 ../certs/server.crt ../certs/server.key
```

步驟 3：運行用戶端

開啟另一個終端機視窗，開始推送 H.265 測試影片：

```
cd build
./moq_client 127.0.0.1 4433 input.h265 2 1200
```

- `2`：發送通道數量（Stream 0 傳送重要影格，Stream 1 傳送一般 P 影格）。
- `1200`：應用層最大載荷切片大小（Bytes）。

步驟 4：驗證輸出檔案完整性

使用 `cmp` 或 `md5sum` 比對傳輸完成後的輸出檔案與原始輸入檔案，兩者內容必須完全一致：

```
# 等待用戶端印出 "Client finished cleanly" 後執行：
cmp input.h265 output_test.h265
echo $? # 若回傳 0，代表兩檔案完全相同，傳輸無損
```

2. 跨主機測試

在實際的網路環境（例如 VPN、區域網路）中測試兩台不同物理主機或虛擬機器之間的媒體串流傳輸：

事前準備

- 確認 Host A（伺服器端）與 Host B（用戶端）的 IP 位址（例如：Host A 的 IP 為 192.168.1.10）。
- 確保兩台主機能互相 ping 通，且處於同一個子網路或已建立 VPN 連線。

步驟 1：伺服器端設定 (Host A 假設IP為 192.168.1.10)

在 Host A 上產生憑證後，啟動伺服器端監聽 0.0.0.0（所有網路介面）：

```
# 綁定監聽所有網路介面
./moq_server 0.0.0.0 4433 output_test.h265 certs/server.crt certs/server.key
```

步驟 2：用戶端設定 (Host B)

在 Host B 上啟動用戶端，並將連線 IP 指向 Host A 的位址：

```
./moq_client 192.168.1.10 4433 input.h265 2 1200
```

步驟 3：驗證傳輸檔案

傳輸完成後，將 Host A 上的輸出檔案 output_test.h265 複製回 Host B 並執行比對：

```
scp user@192.168.1.10:/path/to/output_test.h265 ./
cmp input.h265 output_test.h265
```

3. 多重連線併發測試

藉由在不同的 UDP 埠號上運行多個伺服器進程，並同時啟動多個用戶端進行傳輸。

步驟 1：啟動多個伺服器監聽不同埠號

在後台或使用不同的終端機分頁，啟動多個伺服器實例，分別指定不同的埠號與輸出檔名：

```
cd build

./moq_server 0.0.0.0 4433 output_33.h265 ../certs/server.crt ../certs/server.key &
./moq_server 0.0.0.0 4434 output_34.h265 ../certs/server.crt ../certs/server.key &
```

步驟 2：同時啟動多個用戶端進行傳輸

在用戶端終端機中，同時執行多個用戶端實例，分別連向對應的伺服器埠號：

```
cd build

./moq_client 127.0.0.1 4433 input.h265 2 1200 &
./moq_client 127.0.0.1 4434 input.h265 2 1200 &
```

步驟 3：驗證多重傳輸檔案完整性

當用戶端進程都結束後，驗證多個輸出檔案是否皆與原始輸入檔案完全一致：

```
cmp input.h265 output_33.h265
cmp input.h265 output_34.h265
```

(註：測試完成後，可使用 `kill moq_server` 指令統一關閉背景運作中的伺服器進程。)

4. 控制台日誌詳細解析

在程式執行時，用戶端與伺服器端都會印出詳細的狀態日誌，以利追蹤分包傳輸狀況。以下說明各核心日誌的具體物理意義：

用戶端日誌

1. 連線就緒日誌

```
[cli] wait_connected...
[cli] Connected
[cli] Streams ready
```

- 意義：用戶端已順利與伺服器完成 QUIC 握手與 TLS 1.3 密鑰協商，且所需的多個傳送 Streams 通道已建立完畢。

2. 非同步發送完成日誌

```
[cli] SEND_COMPLETE nalu_id=142 frag_idx=3
```

- 意義：MsQuic 已成功將第 142 號 NALU 的第 3 個分片（0-based）推入網路卡的傳送快取區。用戶端此時可安全釋放或重用該分片的本機記憶體。

3. 用戶端關閉日誌

```
Client finished cleanly
Total Application-Layer packets sent: 8251
[cli] SHUTDOWN_COMPLETE
```

- 意義：所有媒體資料已全部送出，且已向伺服器廣播 Graceful Shutdown 關閉所有 Stream。確認收到 Server 端全數 ACK 後，連線安全結束。

伺服器端日誌

1. 解析封包標頭日誌

```
[parse] nalu_id=142 payload_len=1200
```

- 意義：伺服器的流式解析器（Stream Parser）從 QUIC Stream 讀取出一個 16-byte 的 `PacketHeader`，解析出此網路載荷屬於第 142 號 NALU，其載荷大小為 1200 位元組。

2. 接收碎片順序日誌

```
[RECV_ORDER] t=1034188129754 nalu=142 frag=3/5 bytes=1200
```

- 意義：重組器收到了第 142 號 NALU 的第 3 個分片（該 NALU 總共有 5 個分片），大小為 1200 bytes。接收時的本機單調時間戳記為 1034188129754 微秒。

3. 重組完成與寫入日誌

```
[srv] about to complete consumed=0 total=5640  
[srv] COMPLETE CALLED
```

- 意義：第 142 號 NALU 的所有 5 個分片已全部收齊。重組器成功將它們拼接為一塊連續的 5640 位元組記憶體區塊，並寫入 (Flush) 至磁碟檔案中。

4. 連線中斷與釋放日誌

```
[srv] PEER_SEND_SHUTDOWN stream=0x7f03a4002700  
[srv] SHUTDOWN_COMPLETE
```

- 意義：伺服器收到了用戶端針對該通道發送的 FIN 標記（代表資料傳送結束），伺服器端隨即釋放連線控制資源。

將控制台日誌儲存至文字檔

若您需要將終端機輸出儲存到文字檔以供事後分析，可以使用以下兩種方式：

- 方法 A：重導向至檔案（主畫面不顯示）

使用 `>` 運算子將輸出與錯誤訊息（`2>&1`）一併寫入指定檔案：

```
./moq_client 127.0.0.1 4433 input.h265 2 1200 > client.log 2>&1
```

- 方法 B：同時寫入檔案並在螢幕顯示

利用 `tee` 工具來在即時監視傳輸日誌的同時，於背景將其寫入檔案：

```
./moq_client 127.0.0.1 4433 input.h265 2 1200 2>&1 | tee client.log
```

5. 網路流量擷取、解密與 Wireshark 驗證指南

直接使用 Wireshark 抓包雖然能辨識出 QUIC 協定，但由於其強制使用 TLS 1.3 加密，在未解密的情況下無法直接查看內部的 Stream Data 載荷。本指引介紹如何使用 Wireshark 與 `tshark` 驗證 H.265-over-MsQuic 串流系統，確保自訂的 `PacketHeader` 與影像資料有被正確送上網路。

以 Windows 版 Wireshark 為例。

0. 安裝 Lua Plugin

由於我們的 Header 是自訂格式，必須安裝專屬的 Lua 解析器 (`moq_dissector.lua`) 才能讓 Wireshark 看懂。

1. 打開 Windows 的 Wireshark。
2. 點擊最上方選單的 `Help (說明)` -> `About Wireshark (關於 Wireshark)`。
3. 切換到 `Folders (資料夾)` 分頁。
4. 找到 `Personal Lua Plugins (個人 Lua 外掛程式)` 這一行，對著路徑連點兩下，Windows 會自動打開該資料夾。
5. 在該資料夾內新增一個檔案，命名為 `moq_dissector.lua`。
6. 將虛擬機裡 `~/.config/wireshark/plugins/moq_dissector.lua` 的內容全部複製貼上。
7. 在 Wireshark 中按下 `Ctrl+Shift+L` (或重新啟動程式) 來重新載入外掛。

步驟 1：準備與啟動背景抓包 (終端機 A)

在伺服器端，於背景啟動 Server，並同時準備 `tshark` 背景抓包：

```
cd build

# 1. 背景啟動 Server (必要時，可加入 LD_LIBRARY_PATH 指向偵錯版 msquic)
LD_LIBRARY_PATH=~/.msquic/build/bin/Debug ./moq_server 0.0.0.0 4433 /tmp/out.h265 ../certs/server.crt ..

# 2. 準備 tshark 背景抓包
# (注意：若連線至外網如學校電腦，請將 lo 換成實體介面如 eth0 或 any)
sudo tshark -i lo -f "udp port 4433" -w /tmp/ultimate_capture.pcap
```

步驟 2：執行 Client 傳輸資料並匯出金鑰 (終端機 B)

在用戶端執行 `moq_client` 時，透過環境變數 `SSLKEYLOGFILE` 匯出 TLS 解密金鑰：

```
cd build
rm -f /tmp/ultimate_keys.log

# 執行 Client 並匯出金鑰檔 (請注意正確的命令參數順序)
LD_LIBRARY_PATH=~/.msquic/build/bin/Debug \
SSLKEYLOGFILE=/tmp/ultimate_keys.log \
./moq_client 127.0.0.1 4433 input.h265 2 1200
```

這會迫使 TLS 函式庫將連線金鑰寫入 `/tmp/ultimate_keys.log`。

步驟 3：停止抓包與取得檔案

傳輸完成後，於終端機 A 結束抓包並關閉 Server：

```
# 1. 按 Ctrl+C 停止 tshark
# 2. 結束背景的 moq_server
pkill -f moq_server
```

完成後，請將產生的以下兩個檔案傳送到您的分析電腦：

- `/tmp/ultimate_capture.pcap` (網路封包檔)
- `/tmp/ultimate_keys.log` (TLS 解密金鑰檔)

Wireshark 驗證方法

1. 載入金鑰與解密

- 在本機打開 Wireshark。
- 設定 TLS 金鑰：點擊上方選單 **編輯 (Edit)** -> **偏好設定 (Preferences)** -> **Protocols** -> **TLS**。
- 在 **(Pre)-Master-Secret log filename** 欄位中，選取剛才匯出的 `ultimate_keys.log` 檔案。
- 打開 `ultimate_capture.pcap`。此時點擊 QUIC 封包，應該要能看到底下的 **Protected Payload (decrypted)** 或是 **Stream Data**，代表解密成功。

2. QUIC Stream 切碎與對齊注意事項

由於 QUIC 是連續位元流 (Byte Stream) 協定，而底層有實體 MTU 限制，我們設定的應用層分片在傳輸時會被切碎並與 QUIC 的 STREAM 幀產生錯位：

- 只有當 `PacketHeader` 剛好落在 QUIC `STREAM` 幀的最起點（即 `offset=0` 或剛好對齊的位置），自訂的 Lua 解析插件才能成功抓取並解析它。
- 大多數時候，16 bytes 的 Header 會被切成兩半，分裝在不同的 UDP 封包中。

3. 查看自訂應用標頭的方法

1. 在 Wireshark 上方 Display Filter 輸入 `moq` 並按 Enter。
2. 留下來的封包就是剛好沒被切斷、對齊最前方的封包。
3. 點開這些封包，您可以在下方詳情窗格直接看到 Lua 插件解析出來的 `MoQ PacketHeader` (包含 `nalu_id`, `payload_len`, `fragment_idx`, `fragment_count` 等資訊)。